



Data Structures & Algorithms

Master the fundamentals: Stack • Queue • Deque • Hashing • Recursion using Python

Today's Journey

01

Recap: Stack, Queue, Deque

Revisit the core linear data structures and their operations

03

Recursion Fundamentals

Understand the concept and develop recursive thinking

02

Hashing in Python

Learn how to map data efficiently using hash functions

04

Real-Life Applications

Explore practical examples and solve practice problems

Stack – The LIFO Principle



What is a Stack?

A linear data structure that follows **Last In First Out (LIFO)** — the last element added is the first one removed.

Real-World Examples

- Stack of plates in a cafeteria
- Browser back button functionality
- Undo/Redo operations in text editors

Stack Operations



push()

Insert an element onto the top of the stack



pop()

Remove and return the top element from the stack



peek()

View the top element without removing it



isEmpty()

Check if the stack has no elements

Python Implementation

```
stack = []  
stack.append(10) # push  
stack.append(20) # push  
stack.pop() # returns 20
```

Stack Time Complexity

$O(1)$

Push Operation

Insert at top

$O(1)$

Pop Operation

Remove from top

$O(1)$

Peek Operation

Access top element

All major stack operations run in **constant time**, making stacks incredibly efficient for LIFO scenarios.



Queue – The FIFO Principle

What is a Queue?

A linear data structure following **First In First Out (FIFO)** — the first element added is the first one removed.

Real-World Examples

- Line at a bank or ticket counter
- Print job scheduling
- Task scheduling in operating systems



Queue Operations



enqueue()

Add an element to the rear of the queue



dequeue()

Remove and return the front element



front()

View the front element without removal

Python Implementation

```
from collections import deque
queue = deque()
queue.append(10) # enqueue
queue.append(20) # enqueue
queue.popleft() # returns 10
```




Deque – Double Ended Queue

A versatile data structure that allows insertion and removal from **both ends** — combining the power of stacks and queues.

Python Example

```
from collections import deque
dq = deque()
dq.append(10) # Add to right
dq.appendleft(5) # Add to left
dq.pop() # Remove from right
dq.popleft() # Remove from left
```


When to Use Which Data Structure



Use Stack When

- Implementing undo/redo functionality
- Handling recursion and function calls
- Processing nested structures (parentheses matching)
- Backtracking algorithms



Use Queue When

- Task scheduling and resource allocation
- Breadth-First Search (BFS) algorithms
- Managing requests in web servers
- Print spooling systems



Use Deque When

- Sliding window problems
- Palindrome checking
- Implementing both stack and queue operations
- Efficient insertions at both ends

Introduction to Hashing



What is Hashing?

Hashing maps data of arbitrary size to fixed-size values using a **hash function**. This enables direct access to data without searching.

Key Benefits

- **Fast searching** in $O(1)$ average time
- **Fast insertion** and deletion operations
- Efficient data retrieval using keys

Python's Hash-Based Data Structures

Dictionaries (dict)

Key-value pairs with instant lookups

```
student = {  
    'name': 'Adeel',  
    'age': 24,  
    'grade': 'A'  
}
```

Sets (set)

Unique collections with efficient membership testing

```
unique_numbers = {1, 2, 3, 4, 5}
```

$O(1)$

Average Access Time

Constant time lookups make hash tables incredibly efficient for most operations

How Hashing Works

A hash function transforms keys into array indices, creating a direct mapping between data and storage location.

01

Key Input

Dictionary key is passed to the hash function

02

Hash Generation

Hash function computes a unique numerical value

03

Index Mapping

Hash value is mapped to a specific array index

04

Value Storage

Data is stored at the calculated index position





Hash Collisions

When different keys produce identical hash values, a collision occurs. Python handles these situations automatically using sophisticated strategies.



Chaining

Store multiple values at the same index using linked lists or arrays



Open Addressing

Find the next available slot when a collision occurs through probing techniques

Hashing Time Complexity



Search

$O(1)$ average case

$O(n)$ worst case when collisions occur



Insert

$O(1)$ average case

$O(n)$ worst case during resizing



Delete

$O(1)$ average case

$O(n)$ worst case with collision chains

📌 **Key Insight:** Hash tables provide constant-time performance for most operations, making them ideal for fast data retrieval and storage.

Recursion – Introduction

What is Recursion?

A function that calls itself to solve progressively smaller versions of the same problem.

```
def countdown(n):  
    if n == 0: # Base case  
        print("Done!")  
        return  
    print(n)  
    countdown(n - 1) # Recursive call
```

Essential Components

- Base case to terminate recursion
- Recursive case that calls itself
- Progress toward the base case





Why Use Recursion?

Recursion shines when problems naturally decompose into identical subproblems, making code more elegant and intuitive.



Factorial Calculation

$n! = n \times (n-1)!$ breaks down into smaller factorial problems



Fibonacci Sequence

Each number is the sum of the two preceding numbers



Tree Traversal

Explore branches by recursively visiting child nodes

Structure of Recursion



Base Case

Termination condition that stops recursion

```
def solve(n):  
    # Base case  
    if n == 0:  
        return 0  
  
    # Recursive case  
    return n + solve(n - 1)
```



Recursive Case

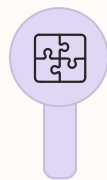
Function calls itself with modified parameters

Pattern Recognition

Identify when a problem can be expressed in terms of itself with smaller input size.



Recursion Thinking Process



Assume Smaller Problem Solved

Trust that the recursive call will handle the reduced version correctly



Use That Solution

Incorporate the result from the recursive call into your current computation



Add Current Step

Combine the recursive result with your current work to solve the larger problem

"Don't try to visualize the entire recursion stack. Write the base case, trust the recursive call, and let the magic happen."

Example – Factorial

Code Implementation

```
def factorial(n):  
    # Base case  
    if n == 1:  
        return 1  
  
    # Recursive case  
    return n * factorial(n - 1)  
  
# Example usage  
result = factorial(4)  
print(result) # Output: 24
```

Execution Flow

1. factorial(4) calls factorial(3)
2. factorial(3) calls factorial(2)
3. factorial(2) calls factorial(1)
4. factorial(1) returns 1
5. Unwind: $2 \times 1 = 2$
6. Unwind: $3 \times 2 = 6$
7. Unwind: $4 \times 6 = 24$

- ❏ **Key Point:** Each recursive call creates a new stack frame, building up until the base case is reached, then unwinding to compute the final result.



Sum of N Numbers

A classic example of recursion that demonstrates how we can break down a problem into smaller, identical subproblems.

```
def sum_n(n):  
    if n == 0:  
        return 0  
    return n + sum_n(n-1)
```

This function calculates the sum of all integers from 1 to n by adding n to the sum of all previous numbers.

Recursion vs Iteration

Recursion

Uses the call stack to manage state

Cleaner, more elegant code

Follows mathematical definitions naturally

Iteration

Uses loops with explicit counters

Often faster and more memory efficient

Can be easier to debug

Recursion and Stack Memory



How It Works

Each recursive call creates a new frame in the call stack, storing local variables and the return address.

The stack follows LIFO (Last In, First Out) principle, unwinding as base cases are reached.

This internal use of stack data structure is what makes recursion elegant but potentially memory-intensive.

Real-Life Recursion Example

01

Folder Contains Files & Subfolders

Each directory holds both immediate files and nested folders

02

Subfolders Contain More

Each subfolder repeats the same structure with its own contents

03

Recursive Structure

This self-similar pattern makes file traversal a perfect recursion problem

Practice Questions (Basic)

Build your foundational recursion skills with these beginner-friendly problems.

1

Reverse String

Reverse a string using recursive approach

2

Palindrome Check

Determine if a number or string reads the same backward

3

Power Function

Calculate x^n using recursion

4

Count Digits

Count the number of digits in an integer

Practice Questions (Intermediate)

Challenge yourself with these more complex recursion problems.

1

Fibonacci Series

Generate Fibonacci numbers recursively

2

Print Range

Print numbers from 1 to n using recursion

3

Array Sum

Calculate sum of array elements recursively

4

Binary Search

Implement binary search using recursive approach

Thinking Question

❏ How Many Recursive Calls?

```
def f(n):  
    if n == 0:  
        return 1  
    return f(n-1) + f(n-1)
```

At each level, the function makes **two** recursive calls. This creates a binary tree of calls.

Time Complexity: $O(2^n)$

Despite its simplicity, this pattern demonstrates exponential growth in recursive calls.



Common Recursion Mistakes

No Base Case

Forgetting to define when recursion should stop leads to infinite recursion and stack overflow errors.

Wrong Base Case

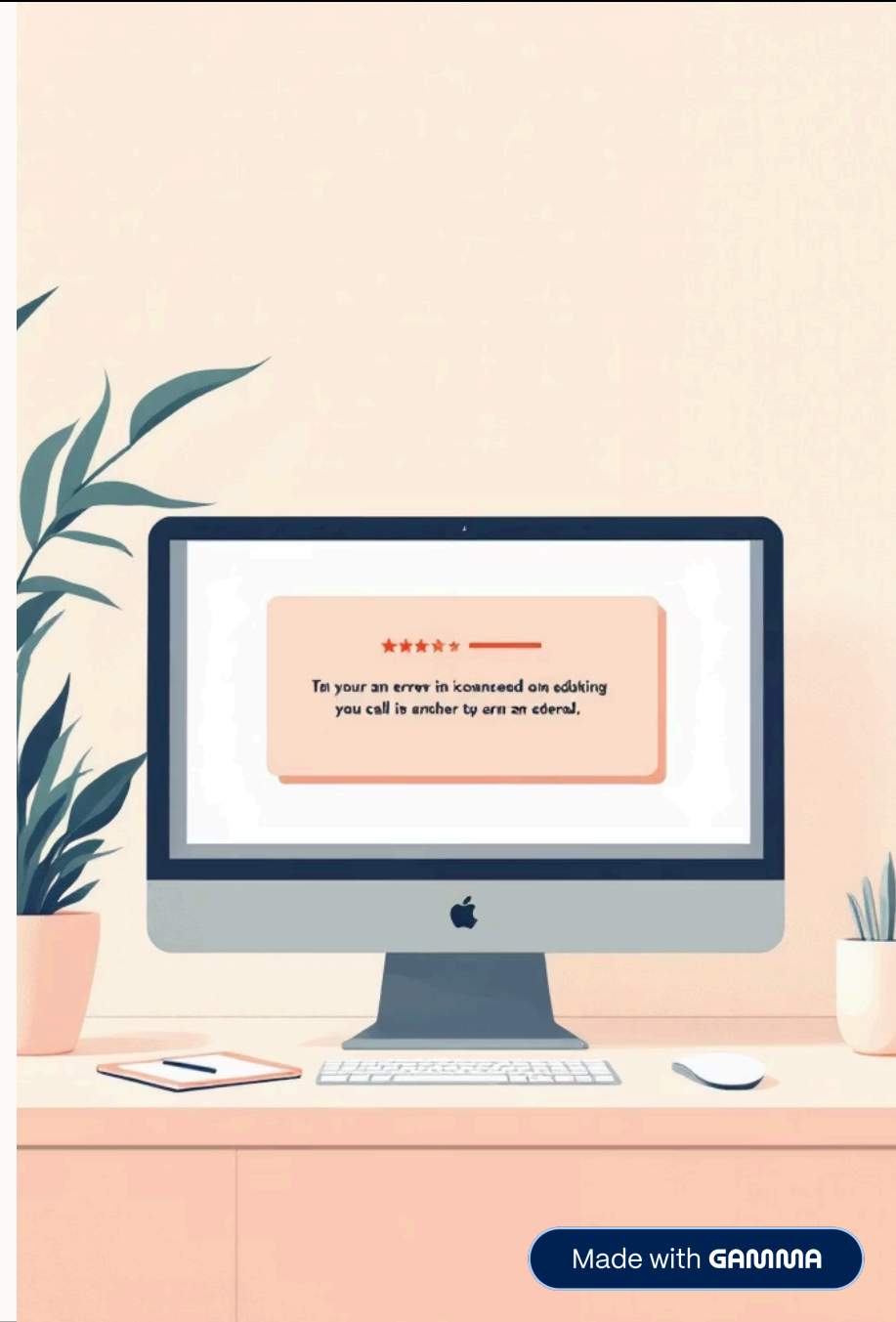
Incorrect base condition means recursion never terminates properly or returns wrong results.

Infinite Recursion

When recursive calls don't converge toward the base case, causing endless execution.

Missing Return

Forgetting to return the result of recursive calls means data gets lost in the call stack.



Summary



Stack – LIFO

Last In, First Out data structure



Queue – FIFO

First In, First Out ordering



Deque – Double Ended

Add and remove from both ends



Hashing – $O(1)$ Access

Constant time lookups



Recursion – Divide & Solve

Break problems into smaller subproblems



Thank You

Questions?

Ready to practice your recursion skills? Start with the basic problems and work your way up to the intermediate challenges.